

Meetbowl 통합 테스트 결과서

1. 테스트 개요

본 문서는 Meetbowl 프로젝트의 각 서비스 레포지토리에서 GitHub Actions를 통해 수행한 통합 테스트 결과를 정리한 문서이다.

Meetbowl은 Frontend, Backend Server, AI Server, STT Server를 분리된 레포지토리로 관리하고 있으며, 각 레포지토리는 `pull_request`, `push`, `workflow_dispatch` 이벤트를 기준으로 GitHub Actions 워크플로우를 실행한다.

각 워크플로우의 `test` 단계에서는 서비스별 실행 환경을 구성한 뒤 자동화 테스트를 수행하며, 테스트가 성공한 경우에만 이후 빌드 및 배포 단계로 진행되도록 구성하였다.

2. 테스트 대상

구분	레포지토리	주요 기술 스택	테스트 방식
Frontend	meetbowl-fe	Vite, Node.js 22	npm test
Backend Server	meetbowl-be	Spring Boot, Java 25, Gradle	./gradlew test
AI Server	meetbowl-ai	FastAPI, Python 3.12, uv	pytest
STT Server	meetbowl-stt	Node.js 22	npm test, npm run build

3. GitHub Actions 테스트 실행 조건

각 서비스의 테스트 워크플로우는 다음 조건에서 실행된다.

이벤트	실행 내용
pull_request → main	테스트 실행
push → main	테스트 실행 후 빌드 및 배포 진행
workflow_dispatch	수동 실행

Pull Request 단계에서는 운영 배포를 수행하지 않고 테스트만 실행하여 main 브랜치 병합 전 코드 안정성을 검증한다.

main 브랜치에 push되거나 수동 실행되는 경우에는 테스트가 통과한 뒤 Docker 이미지 빌드, ECR 업로드, EC2 배포 또는 S3/CloudFront 배포가 진행된다.

4. 서비스별 테스트 구성

4.1 Frontend 테스트 구성

Frontend는 Node.js 22 환경에서 테스트와 빌드를 수행한다.

테스트 단계 구성

1. GitHub 저장소 checkout
2. Node.js 22 환경 구성
3. npm 캐시 설정
4. `npm ci` 명령어로 의존성 설치
5. `npm test` 명령어로 테스트 수행
6. `npm run build` 명령어로 Vite 빌드 검증

테스트 명령어

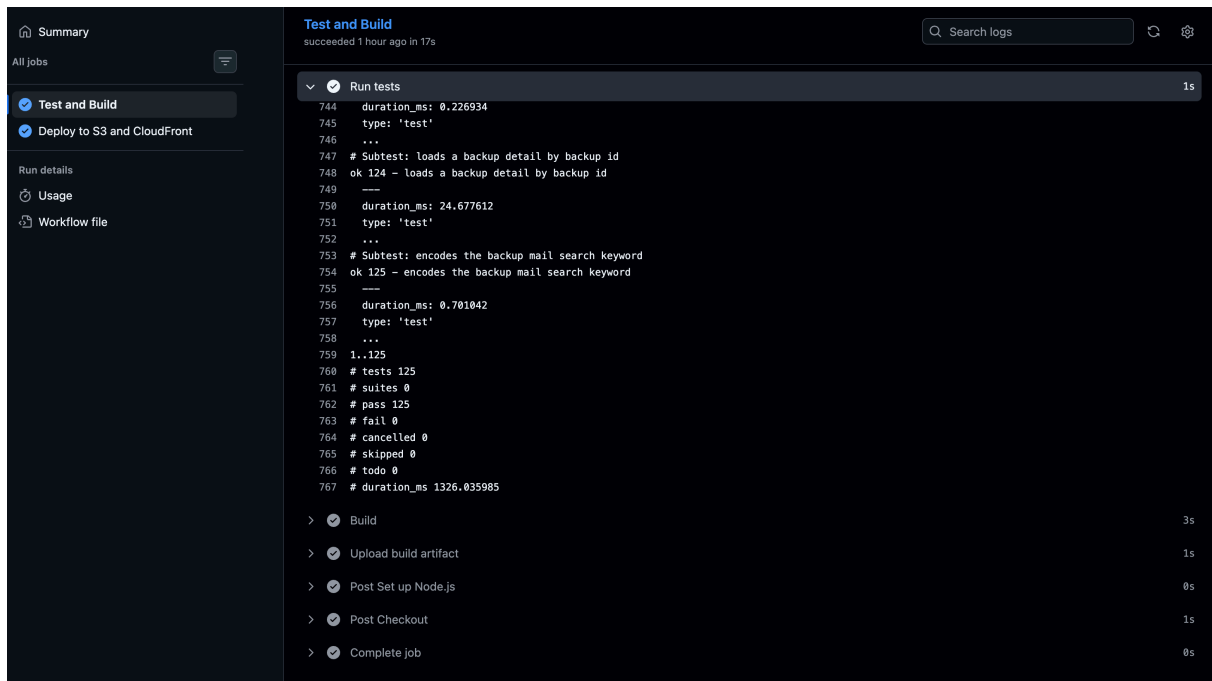
```
npm test  
npm run build
```

검증 내용

Frontend 테스트 단계에서는 프론트엔드 코드가 정상적으로 테스트를 통과하는지 확인하고, Vite 빌드가 성공적으로 수행되는지 검증한다.

이를 통해 배포 전 정적 파일 생성 가능 여부를 확인한다.

테스트 결과



4.2 Backend Server 테스트 구성

Backend Server는 Spring Boot 기반 API 서버이며, GitHub Actions 테스트 환경에서 MariaDB, Redis, RabbitMQ 서비스 컨테이너를 함께 실행한다.

테스트 단계 구성

1. GitHub 저장소 checkout
2. MariaDB 11.8 서비스 컨테이너 실행
3. Redis 7.4 서비스 컨테이너 실행
4. RabbitMQ 3.13-management 서비스 컨테이너 실행
5. Java 25 환경 구성
6. Gradle 캐시 설정
7. `gradlew` 실행 권한 부여
8. 테스트용 환경변수 주입
9. `./gradlew test --no-daemon` 명령어로 테스트 수행

테스트 명령어

```
./gradlew test --no-daemon
```

테스트 인프라

구성 요소	사용 이미지	용도
MariaDB	mariadb:11.8	테스트 DB
Redis	redis:7.4-alpine	캐시 및 실시간 상태 테스트
RabbitMQ	rabbitmq:3.13-management	이벤트 메시징 테스트

검증 내용

Backend 테스트 단계에서는 실제 운영 인프라에 직접 접근하지 않고, GitHub Actions 내부 서비스 컨테이너를 사용하여 API 서버의 테스트를 수행한다.

테스트용 DB, Redis, RabbitMQ, JWT, LiveKit, STT, AI, S3 관련 환경변수를 주입하여 운영 Secret 없이도 통합 테스트가 가능하도록 구성하였다.

이를 통해 Backend Server가 주요 외부 의존성을 가진 상태에서도 정상적으로 테스트를 통과하는지 검증하였다.

테스트 결과

The screenshot displays the GitHub Actions interface for a workflow named 'Test'. The status is 'succeeded 6 minutes ago in 3m 38s'. The left sidebar shows the workflow steps: Summary, All jobs, Test (selected), Build And Push Image, Deploy, Run details, Usage, and Workflow file. The main panel shows the logs for the 'Run backend tests' job, which completed in 2m 35s. The logs include timestamps, log levels (INFO), and messages from the application, such as 'Shutdown initiated...' and 'Shutdown completed...' for various HikariPool instances. The final output shows 'BUILD SUCCESSFUL in 2m 35s' and '22 actionable tasks: 22 executed'. A link is provided to enable the configuration cache for faster builds.

4.3 AI Server 테스트 구성

AI Server는 FastAPI 기반 서버이며, Python 3.12와 uv를 사용하여 테스트 환경을 구성한다.

테스트 단계 구성

1. GitHub 저장소 checkout
2. Python 3.12 환경 구성
3. uv 설치 및 캐시 활성화
4. `uv sync --frozen --dev` 명령어로 의존성 설치
5. 테스트용 환경변수 주입
6. `uv run pytest -q` 명령어로 테스트 수행

테스트 명령어

```
uv sync --frozen --dev
uv run pytest -q
```

테스트 환경변수

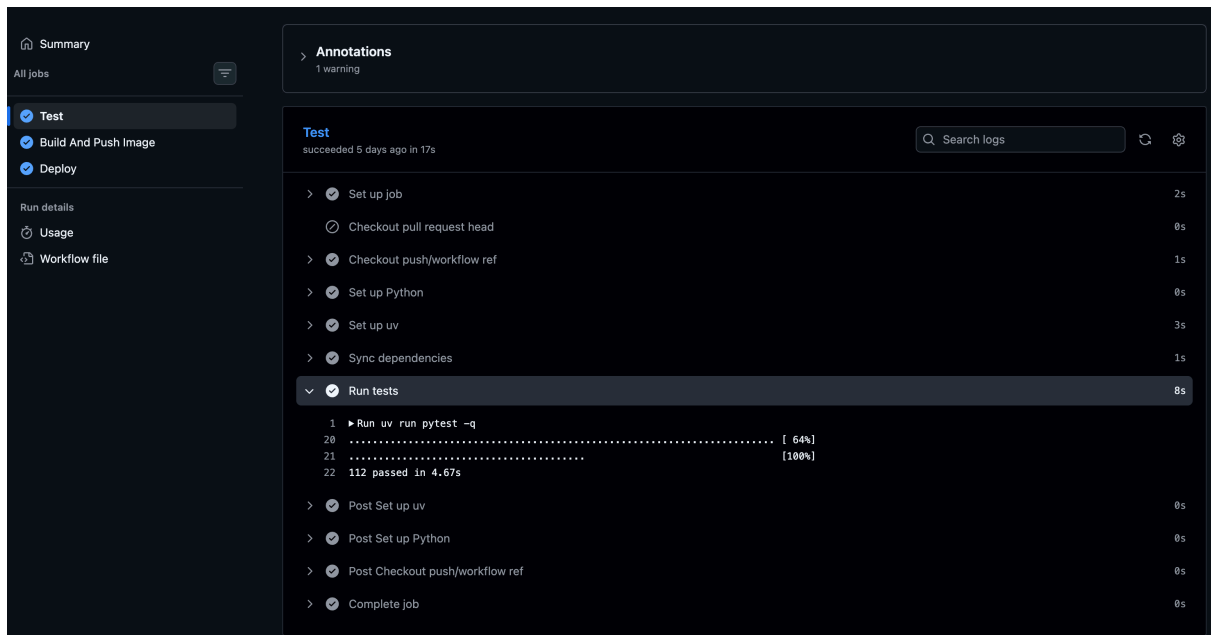
환경변수	용도
INTERNAL_TOKEN	내부 API 인증 테스트
BE_BASE_URL	Backend API 연동 주소
GEMINI_API_KEY	테스트용 Gemini API Key
OPENAI_API_KEY	테스트용 OpenAI API Key

검증 내용

AI Server 테스트 단계에서는 실제 운영 API Key를 사용하지 않고 테스트용 값을 주입하여 테스트를 수행한다.

이를 통해 FastAPI 애플리케이션의 주요 기능과 AI 처리 로직이 CI 환경에서 정상적으로 실행되는지 검증하였다.

테스트 결과



4.4 STT Server 테스트 구성

STT Server는 Node.js 기반 실시간 STT 중계 서버이며, 테스트와 빌드를 함께 수행한다.

테스트 단계 구성

1. GitHub 저장소 checkout
2. Node.js 22 환경 구성
3. npm 캐시 설정
4. `npm ci` 명령어로 의존성 설치
5. `npm test` 명령어로 테스트 수행
6. `npm run build` 명령어로 애플리케이션 빌드 검증

테스트 명령어

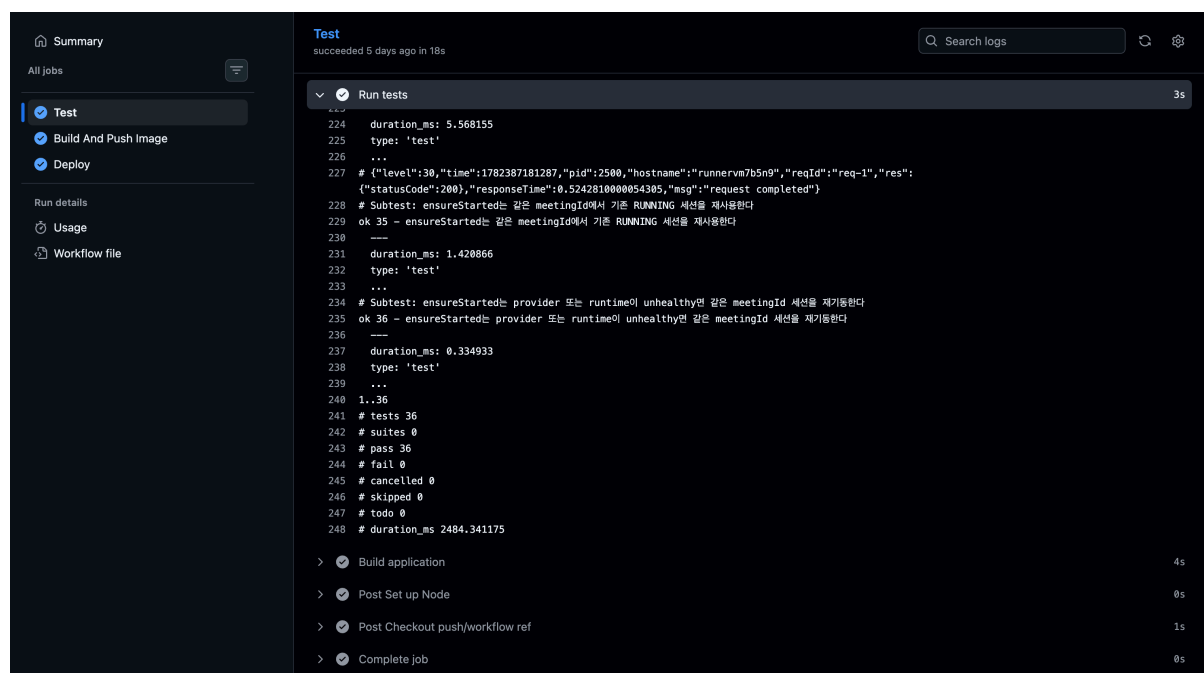
```
npm test
npm run build
```

검증 내용

STT Server 테스트 단계에서는 Node.js 애플리케이션의 테스트가 정상적으로 수행되는지 확인하고, TypeScript 또는 빌드 대상 코드가 정상적으로 빌드되는지 검증한다.

이를 통해 실시간 STT 서버가 배포 가능한 상태인지 확인하였다.

테스트 결과



5. 테스트 결과 상세

5.1 Frontend

Frontend 워크플로우에서는 Node.js 22 환경을 구성한 뒤 의존성 설치, 테스트, 빌드를 순차적으로 수행하였다.

`npm test` 와 `npm run build` 가 정상적으로 완료되었으며, 빌드 결과물인 `dist` 디렉토리가 생성되었다.

결과적으로 Frontend는 배포 가능한 정적 파일 생성까지 정상적으로 검증되었다.

5.2 Backend Server

Backend Server 워크플로우에서는 MariaDB, Redis, RabbitMQ를 GitHub Actions 서비스 컨테이너로 실행한 뒤 Spring Boot 테스트를 수행하였다.

테스트 환경변수를 통해 DB, Redis, RabbitMQ, JWT, LiveKit, STT, AI, S3 관련 설정을 주입하였고, `./gradlew test --no-daemon` 명령어가 정상적으로 완료되었다.

결과적으로 Backend Server는 주요 인프라 의존성을 포함한 테스트 환경에서 정상 동작함을 확인하였다.

5.3 AI Server

AI Server 워크플로우에서는 Python 3.12와 uv 기반 환경을 구성한 뒤 `uv sync --frozen -dev` 로 의존성을 설치하였다.

이후 테스트용 내부 토큰과 API Key를 주입하고 `uv run pytest -q` 명령어를 실행하였다.

테스트가 정상적으로 완료되어 AI Server의 주요 기능이 CI 환경에서 검증되었다.

5.4 STT Server

STT Server 워크플로우에서는 Node.js 22 환경에서 의존성 설치, 테스트, 빌드를 수행하였다.

`npm test` 와 `npm run build` 가 정상적으로 완료되었으며, STT Server가 배포 가능한 상태임을 확인하였다.

6. 결론

Meetbowl의 각 서비스는 GitHub Actions 기반 CI 환경에서 자동 테스트를 수행하도록 구성되어 있다.

Frontend, Backend Server, AI Server, STT Server 모두 test 단계가 정상적으로 통과하였으며, 테스트 성공 이후에만 빌드 및 배포 단계로 진행되는 구조를 갖는다.

이를 통해 main 브랜치에 반영되는 코드의 기본 동작 안정성을 검증하고, 배포 전 오류를 조기에 발견할 수 있도록 하였다